MODULE 1

MCA203 Design and Analysis of Algorithms



ALGORITHMS

- An algorithm is a sequence of **computational steps** that transform the **input** into **the output**

ALGORITHM VS PROGRAM

```
BUBBLESORT(A)
1  for i = 1 to A.length - 1
2      for j = A.length downto i + 1
3          if A[j] < A[j - 1]
4              exchange A[j] with A[j - 1]
```

CHARACTERISTICS OF ALGORITHM

- Input – 0 or more
- Output –atleast 1
- Definiteness-every statement shd be unambiguous and solvable
- Effectiveness-every statement should have a purpose
- Finiteness-finite no of steps ,it stops and returns output

Table 1.1 The number of comparisons done by Sequential Search and Binary Search when x is larger than all the array items

Array Size	Number of Comparisons by Sequential Search	Number of Comparisons by Binary Search
128	128	8
1,024	1,024	11
1.048,576	1,048,576	21
4,294,967,296	4,294.967,296	33

Frequency count: Examples

- Algorithm_swap(a,b)
- {
  - temp=a; 1
  - a=b; 1
  - B=temp; 1
- } total: 3
-
- Time : O(1) ie constant
- Space :temp,a,b, : O(1) ie constant

Examples

- Algorithm_sum(A,n)
- { s=0: …… 1
  - for(i=0;i<n;i++) …… N+1
- s=s+A[i]; ……n
- return s; ……1
- }

- Time :2n+3 ie o(n)
- Space: A,s,n,i=n+1+1+1=n+3= ie o(n)

- Algorithm_matrix_add(A,B,n)
  - {
- for(i=0;i<n;i++) ……n+1
- {
  For(j=0;j<n;j++) ……n(n+1)
  {

  c[i][j]=A[i][j]+B[i][j];……n*n
  }
- }
- }

- Time: (n+1) + (n(n+1)) + n*n =$2n^2$ + 2n+ 1=>On^2)
- Space : i,n,j,A,B,C=>1+1+1+n^2 + n^2 +n^2 =$3n^2$ +3=> O(n^2)

- Algorithm_matrix_mul(A,B,n)
  - {
- for(i=0;i<n;i++) ……n+1
- {
  - For(j=0;j<n;j++) ……n(n+1)
  - {
    - c[i][j]=0; …… n*n
    - For(k=0;j<n;j++) ……(n+1)* n*n
    - {A[i][j]=B[i][j]+C[i][j];……n*n*n
    - }
  - }
- }
- }
- Time :$O(n^3)$
- Space: $O(n^2)$

EFFICIENCEY OF ALGORITHMS

When analyzing the efficiency of an algorithm in terms of time, we do not determine the actual number of CPU cycles, because this depends on the particular computer on which the algorithm is run. Furthermore, we do not even want to count every instruction executed, because the number of instructions depends on the programming language used to implement the algorithm and the way the programmer writes the program. Rather, we want a measure that is independent of the computer, the programming language, the programmer, and all the complex details of the algorithm such as incrementing of loop indices, setting of pointers,

In general, the running time of an algorithm increases with the size of the input, and the total running time is roughly proportional to how many times some basic operation (such as a comparison instruction) is done. We therefore analyze the algorithm's efficiency by determining the number of times some basic operation is done as a function of the size of the input.

In the analysis of linear and binary search

comparing the numbers of comparisons done by the two algorithms for various values of n, where n is the number of items in the array.